

Machine Learning Lab Schedule

Experiments	Content	Week
1	KNN Algorithm for Iris Flower Classification	Week 6 Firday: 6:45-10:05 PM Room T2510 
2	Classification of Wine Dataset Using PCA and SVM Algorithms	Week 8 Thursday: 10:30-12:15AM Thursday: 4:00-17:45 PM Room T2510 
 3	CNN for MNIST Digit Recognition	Week 10 Wednesday: 4:00-17:45 PM Thursday: 4:00-17:45 PM Room T2510 



Experiment 3

CNN for MNIST Digit Recognition

2025

Objective

- Gain a basic understanding of CNN architectures
- Learn how to implement CNN network using Python with PyTorch
- Evaluate the model's performance in accurately classifying images of handwritten digits



Environment

➤ Python environment

- Anaconda & Jupyter Notebook



➤ Install PyTorch and Torchvision

```
conda activate base  
pip install torch torchvision torchaudio
```

In Anaconda Prompt

<https://pytorch.org/get-started/locally/>

➤ Import required libraries

```
import torch  
import torchvision  
from torch.utils.data import DataLoader
```

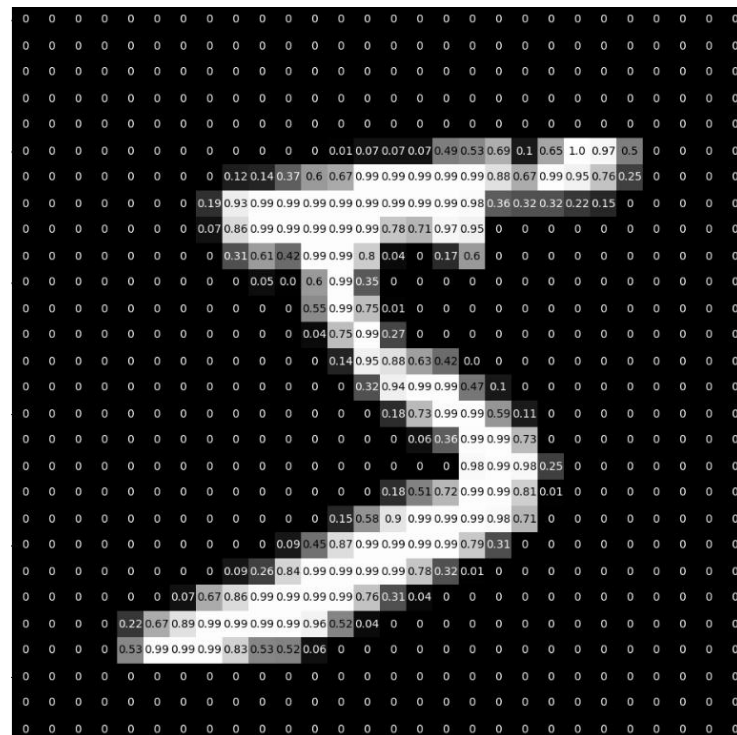
Experiment Tasks

1. Load MNIST Dataset

The MNIST dataset is a large collection of handwritten digit images.

It consists of **60,000** training images and **10,000** testing images.

Each image is a **28x28 pixel** grayscale image, representing a single digit from **0 to 9**.



Experiment Tasks

2. Configure Training Parameters

Number of epochs to train the model

`n_epochs = 3`

Batch size for training data & testing data

`batch_size_train = 64`

`batch_size_test = 1000`

Learning rate for the optimizer

`learning_rate = 0.01`

Momentum value for the optimizer (if using SGD with momentum)

`momentum = 0.5`

Interval for logging training statistics

`log_interval = 10`

Random seed for reproducibility

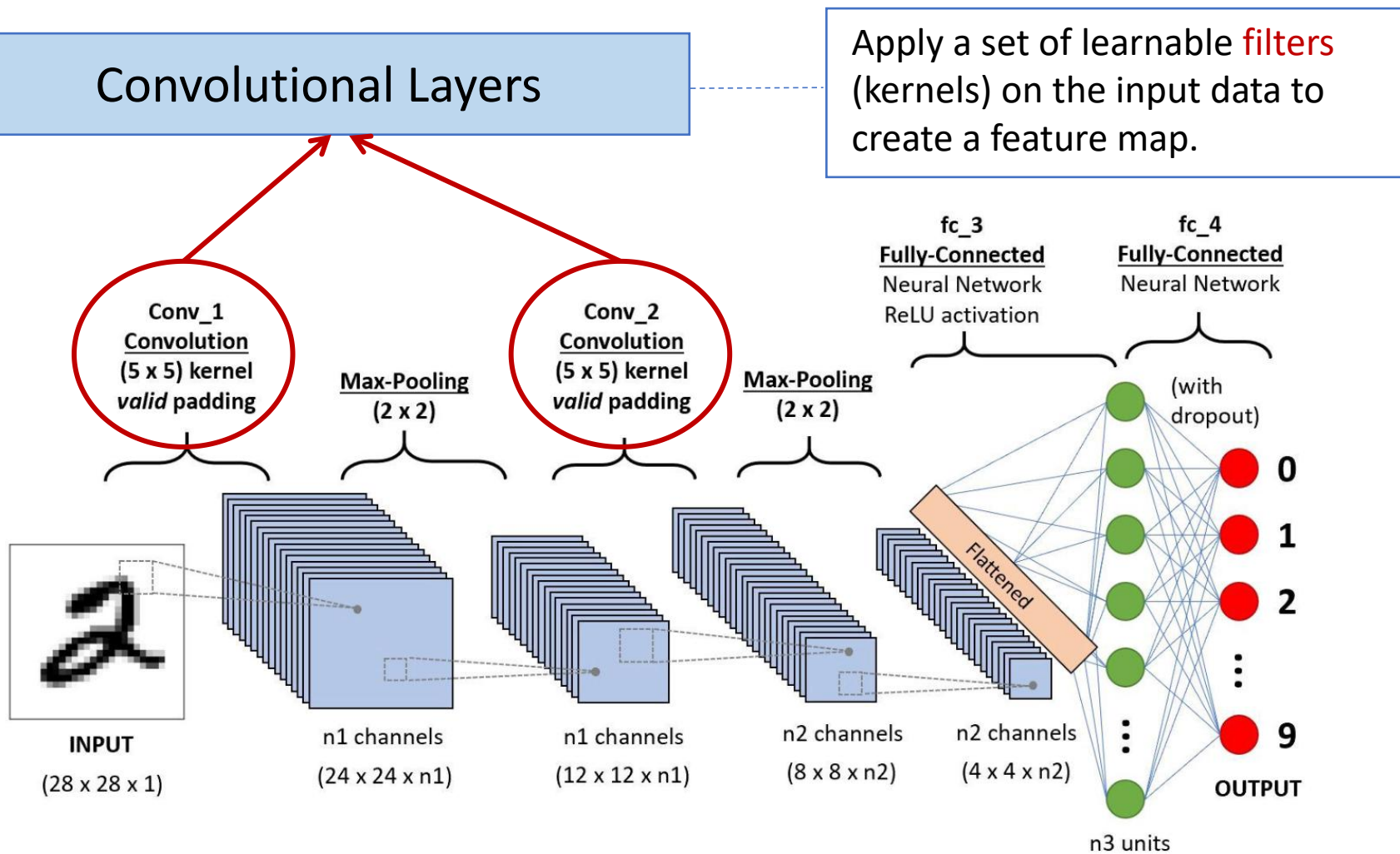
`random_seed = 1`

`torch.manual_seed(random_seed)`

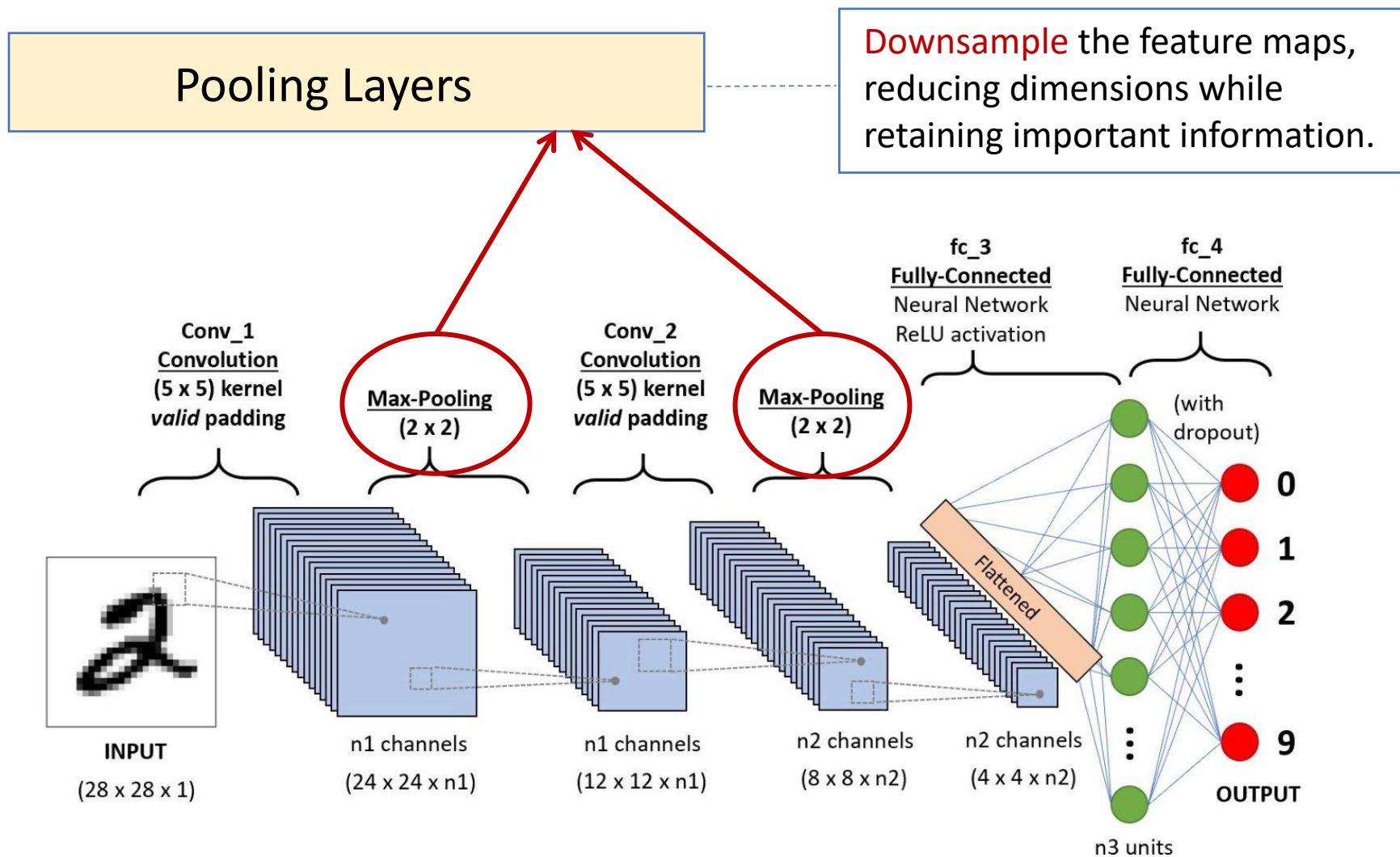
hyperparameters

Experiment Tasks

3. Define the CNN Architecture

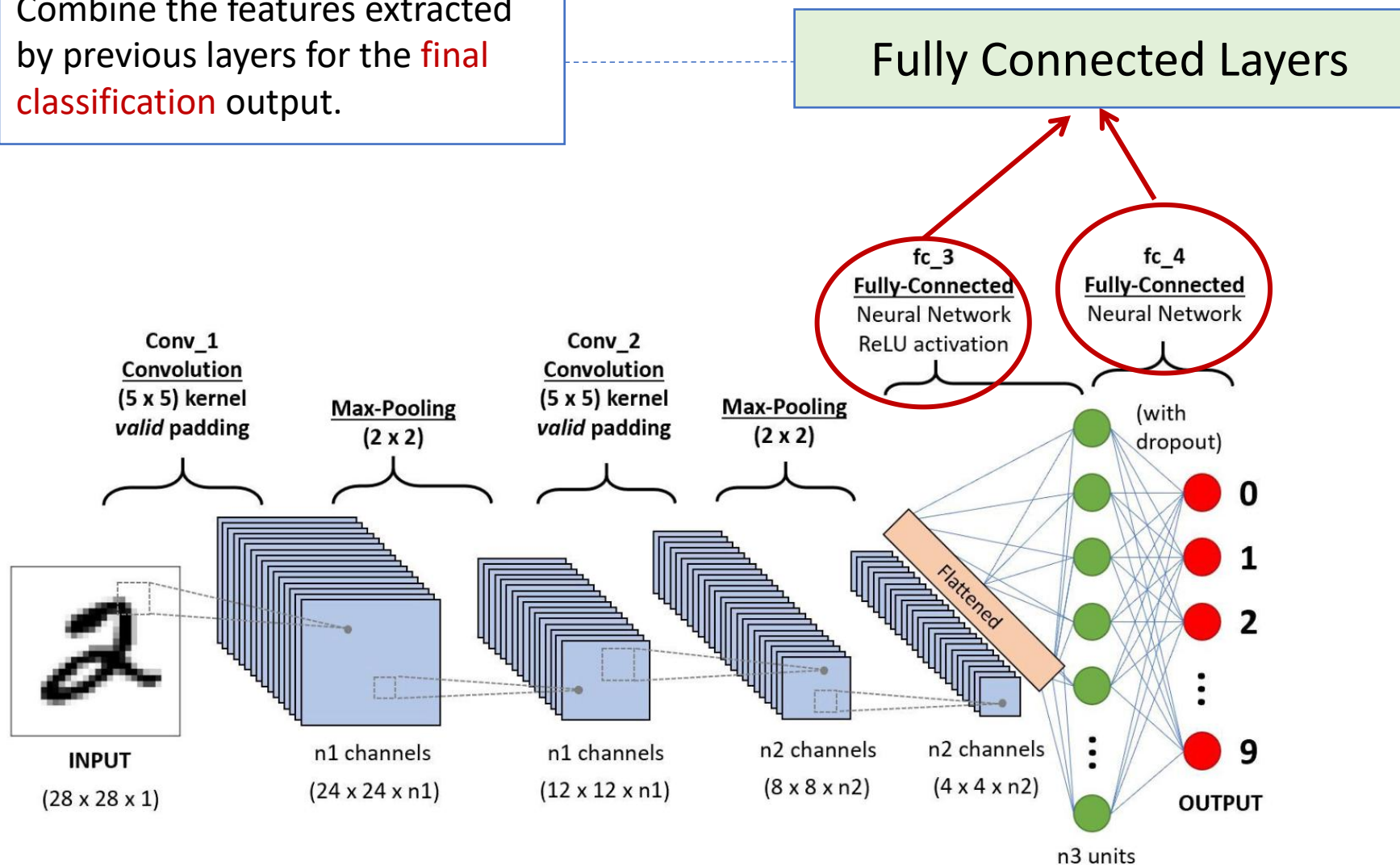


Experiment Tasks



Experiment Tasks

Combine the features extracted by previous layers for the **final classification** output.



Experiment Tasks

Define CNN Architecture

Import the neural network module from Torch

```
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
```

```
class Net(nn.Module):
```

Define the structure of the neural network

```
def __init__(self):
```

```
    super(Net,self).__init__()
    self.conv1 = nn.Conv2d(1, 10, kernel_size=5)
    self.conv2 = nn.Conv2d(10, 20, kernel_size=5)
    self.conv2_drop = nn.Dropout2d()
    self.fc1 = nn.Linear(320, 50)
    self.fc2 = nn.Linear(50, 10)
```

Define the forward pass of the network

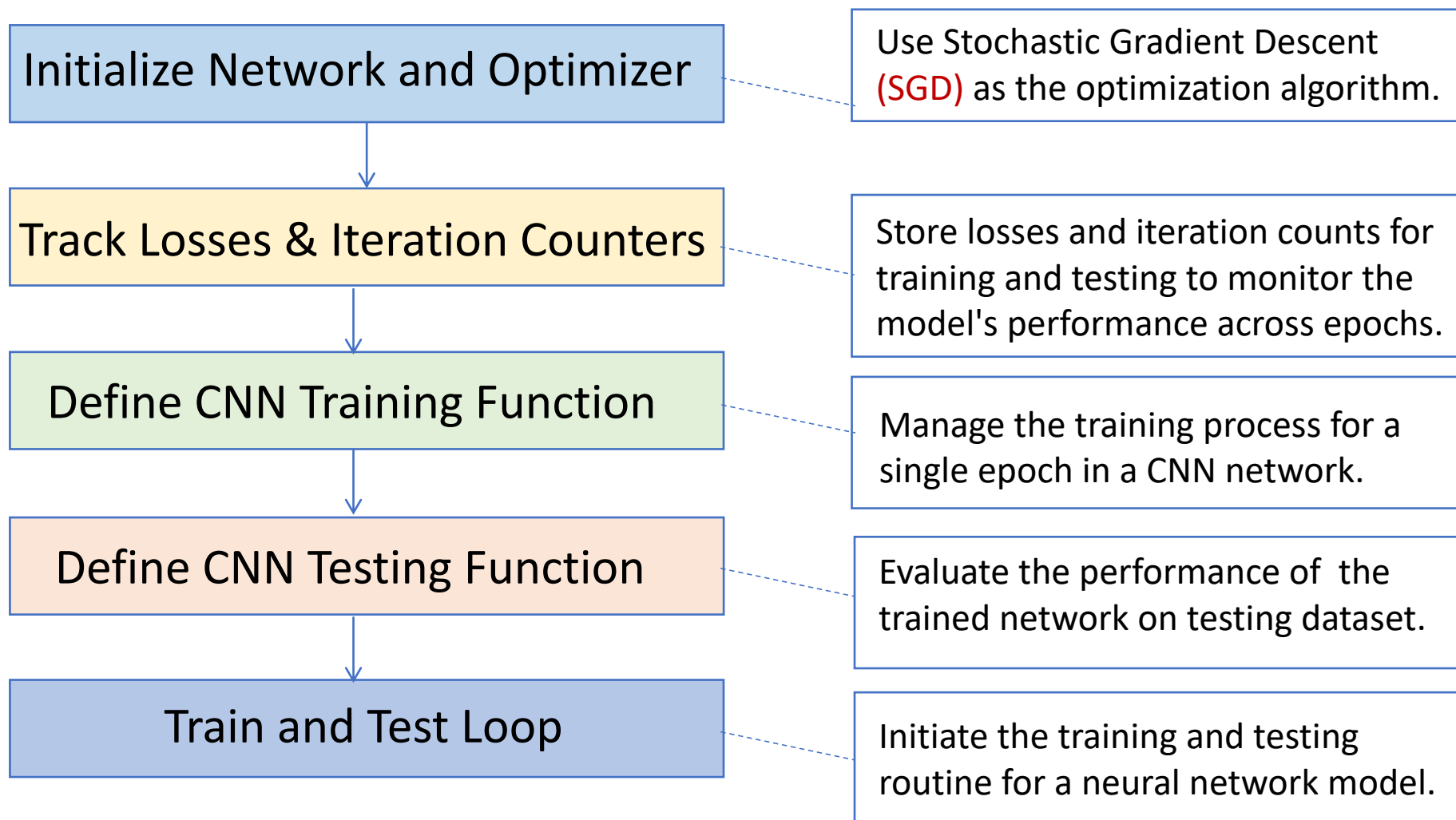
```
def forward(self, x):
```

```
    x = F.relu(F.max_pool2d(self.conv1(x), 2))
    x = F.relu(F.max_pool2d(self.conv2_drop(self.conv2(x)), 2))
    x = x.view(-1, 320)
    x = F.relu(self.fc1(x))
    x = F.dropout(x, training=self.training)
    x = self.fc2(x)
    return F.log_softmax(x,dim=1)
```

IMPORTANT:
Pay attention to indentation!

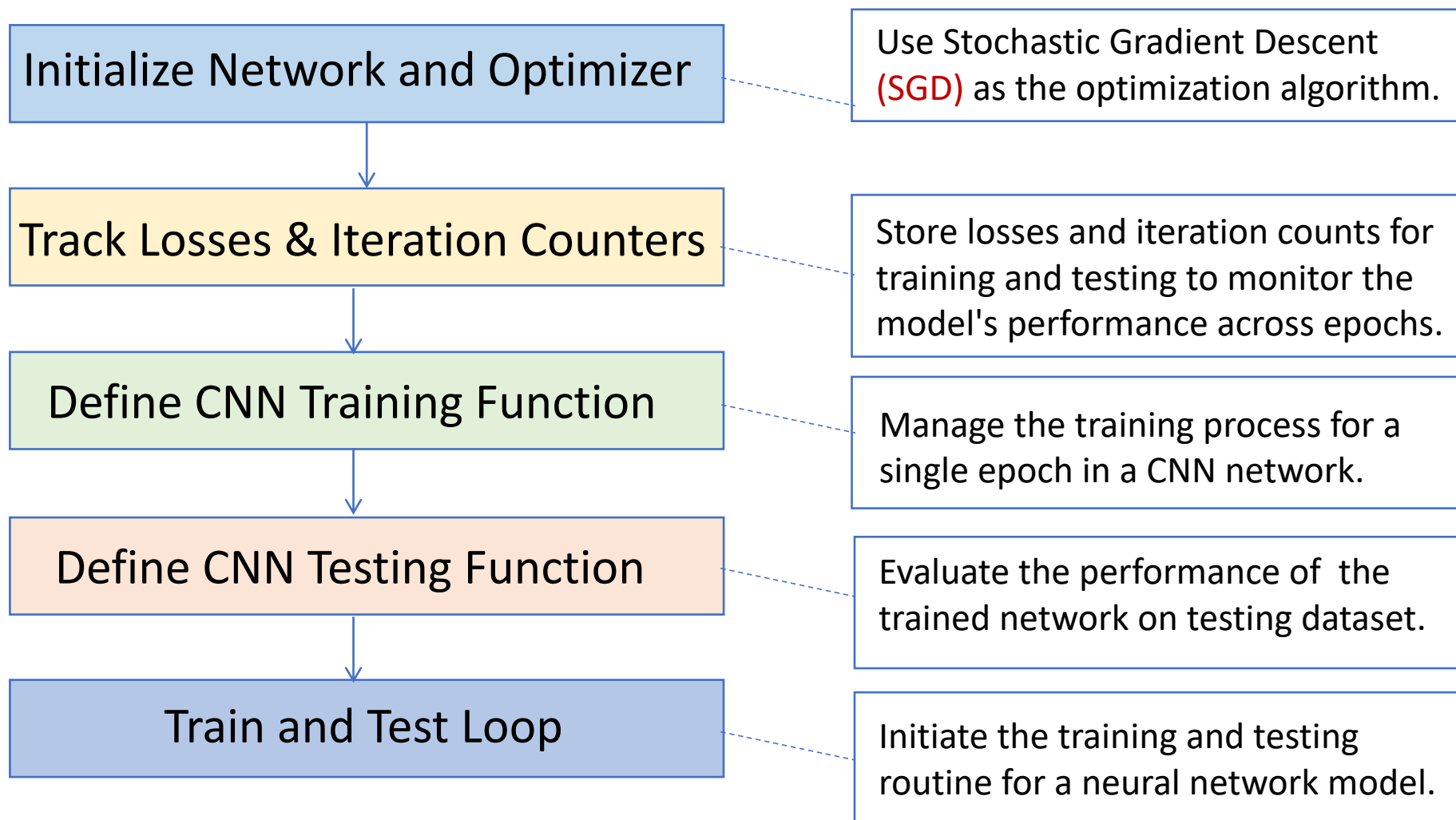
Experiment Tasks

4. Model Training and Evaluation



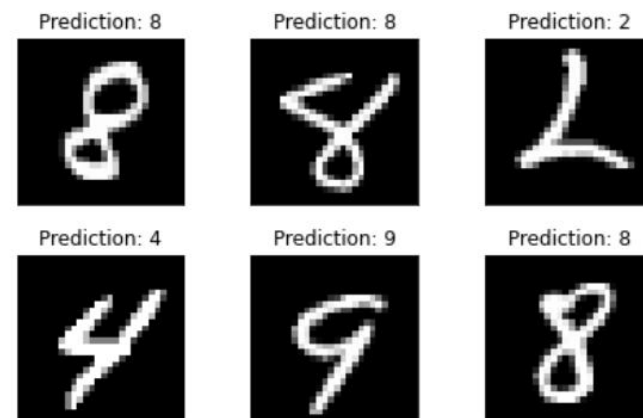
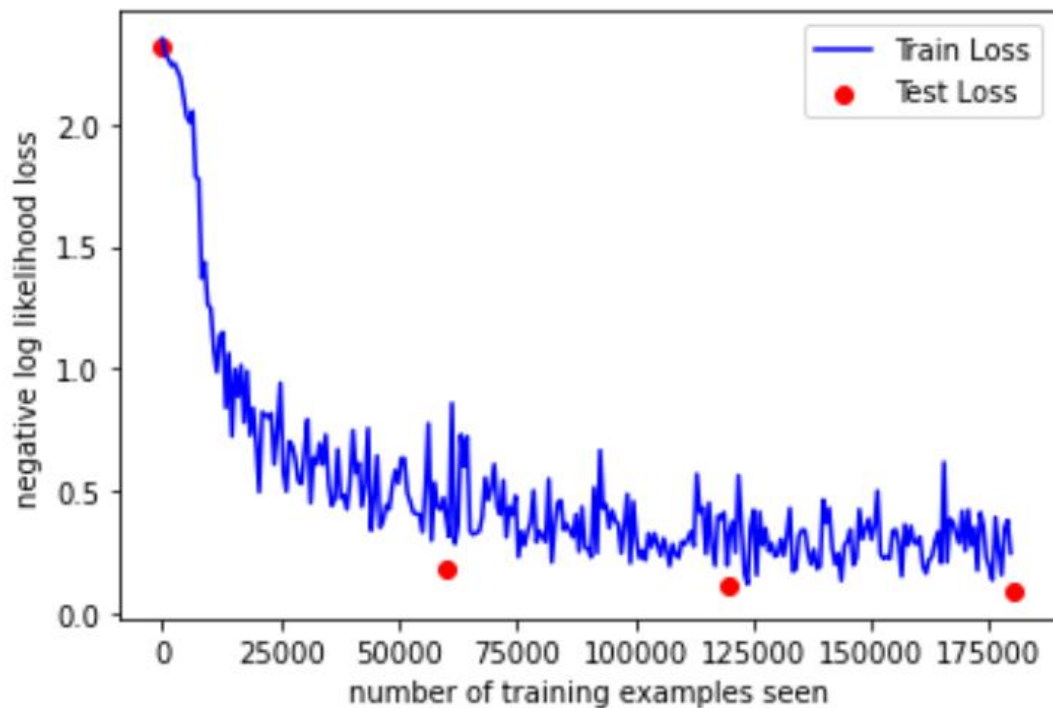
Experiment Tasks

4. Model Training and Evaluation



Experiment Tasks

5. Data Monitoring and Visualization



Note: If you encounter a kernel crash when drawing graphs on the lab computer, you can try running the following commands.

```
import os
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
```



THANK YOU!